Through WebAssembly (Node.js)

Impulses can be deployed as a WebAssembly library. This packages all your signal processing blocks, configuration and learning blocks up into a single package. You can include this package in web pages, or as part of your Node.js application. This allows you to run your impulse locally, without any compilation. In this tutorial you'll export an impulse, and build a Node.js application to classify sensor data. You can also load this library from a web page, see Through WebAssembly (browser).

Prerequisites

Make sure you followed the Continuous motion recognition tutorial, and have a trained impulse. Also install the following software:

- Node.js - to build the application.

Deploying your impulse

Head over to your Edge Impulse project, and go to **Deployment**. From here you can create the full library which contains the impulse and all external required libraries. Select **WebAssembly** and then click **Build** to create the library. Download and unzip the `.zip` file.

Then, create a new file called `run-impulse.js` in the same folder, and add:

run-impulse.js

```
let jsResult = {
    anomaly: ret.anomaly,
    results: []
};

// if ret.size is a function, then this is a new module. Use this API call to prevent leaks.
// the old API (calling via ret.classification) is still there for backwards compatibility, but
if (typeof ret.size === 'function') {
    for (let cx = 0; cx < ret.size(); cx++) {
        let c = ret.get(cx);
        jsResult.results.push({ label: c.label, value: c.value });
        c.delete();
    }
}
else {
    for (let cx = 0; cx < ret.classification.size(); cx++) {
        let c = ret.classification.get(cx);
        jsResult.results.push({ label: c.label, value: c.value });
        c.delete();
    }
}

ret.delete();
```

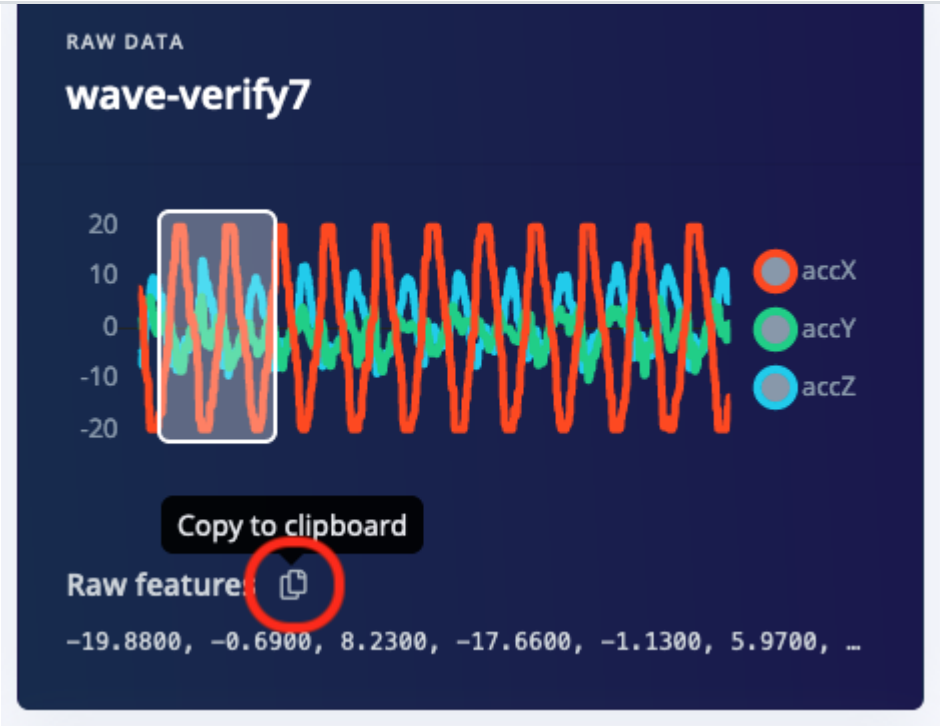
```
            let ptr = Module._malloc(numBytes);
            let heapBytes = new Uint8Array(Module.HEAPU8.buffer, ptr, numBytes);
            heapBytes.set(new Uint8Array(typedArray.buffer));
            return { ptr: ptr, buffer: heapBytes };
        }
    }

    if (!process.argv[2]) {
        return console.error('Requires one parameter (a comma-separated list of raw features, or a file poin
    }

    let features = process.argv[2];
    if (fs.existsSync(features)) {
        features = fs.readFileSync(features, 'utf-8');
    }

    // Initialize the classifier, and invoke with the argument passed in
    let classifier = new EdgeImpulseClassifier();
    classifier.init().then(async () => {
        let result = classifier.classify(features.trim().split(',').map(n => Number(n)));

        console.log(result);
    }).catch(err => {
        console.error('Failed to initialize classifier', err);
    });
```

Running the impulse

With the project ready it's time to verify that the application works. Head back to the studio and click on **Live classification**. Then load a validation sample, and click on a row under 'Detailed result'.

TIMESTAMP	IDLE	SNAKE	UPDOWN	WAVE	ANOMALY
0	0	0	0	0.99	0.42
80	0.04	0	0.01	0.95	0.35
160	0	0	0	0.99	0.25
240	0	0	0	0.99	0.19
320	0.02	0	0	0.98	0.13
400	0	0	0	0.99	0.09

Selecting the row with timestamp '320' under 'Detailed result'.



Copying the raw features

Then invoke the local application by passing these features as an argument to the application. Open a terminal or command prompt and run:

```
node run-impulse.js "-19.8800, -0.6900, 8.2300, -17.6600, -1.1300, 5.9700, ..."
```

This will run the signal processing pipeline, and then classify the output:

```
{
  anomaly: 0.133557,
  results: [
    { label: 'idle', value: 0.015319 },
    { label: 'snake', value: 0.000444 },
    { label: 'updown', value: 0.006182 },
    { label: 'wave', value: 0.978056 }
  ]
}
```

Which matches the values we just saw in the studio. You now have your impulse running locally!

Troubleshooting

Argument list too long

There's a limited number of arguments that you can pass on the command line, and if your raw features array is larger than this number you will be presented with the error: `Argument list too long`. To get around this you can create a file called `features.txt`, put your features in there, and then call the script via:

```
node run-impulse.js features.txt
```